

Plan De Rendimiento Ultra Para SaltorSystem

Analisis por pantalla, medidas de reduccion de consumo y recomendacion de infraestructura para operar con mas de 300 usuarios concurrentes.

Corte tecnico: 24/08/2026 - Republica Dominicana. Basado en el reporte de carga observado: PostgreSQL/Supabase DB cerca de 702% CPU, 12 consultas activas, 2,578 solicitudes en 2 minutos, 906 a /rest/v1/factura, 145 timeouts 504 y 65 errores 500.

Resumen Ejecutivo

- El problema principal no es RAM ni disco: es CPU consumida por una tormenta de solicitudes REST, sobre todo contra myappdb.factura, junto con consultas repetidas de usuario, productos2 e inventario.
- Ya se empezaron mejoras en Facturacion, Cotizacion y Caja: listas con columnas especificas, detalle bajo demanda, menos select *, paginacion mas corta y filtros por sucursal.
- Para 300+ usuarios, el objetivo debe ser reducir solicitudes, reducir filas leidas, reducir columnas transferidas y mover resúmenes pesados a RPC SQL con índices.
- La infraestructura debe separarse por capas: frontend estatico/CDN, API/PostgREST, PostgreSQL con pool de conexiones, monitoreo y backups. Subir CPU sin corregir consultas solo aplaza el problema.

Decision Sobre DigitalOcean Vs Self-Hosted Actual

DigitalOcean puede ser mejor que el self-hosted actual si se usa como arquitectura administrada y separada, no simplemente como otro servidor unico corriendo todo. La mejora real vendria de usar Managed PostgreSQL con connection pooling, metricas, backups, failover y recursos dedicados para la base de datos. DigitalOcean documenta PgBouncer/connection pools para cargas con muchas conexiones, metricas de PostgreSQL como conexiones, cache hit ratio y throughput, y planes administrados con failover. Supabase self-hosted sigue siendo valido cuando se necesita control total, pero en produccion exige operar Docker, HTTPS/reverse proxy, monitoreo, backups, logs y tuning de Postgres.

Opcion	Ventaja	Riesgo	Recomendacion
Self-hosted actual	Control total y menor costo directo si el equipo administra bien PostgreSQL.	Si API, DB, auth, logs y jobs comparten CPU, una pantalla pesada calienta todo el servidor.	Mantener solo si se separa DB/API, se agrega pool, observabilidad y tuning serio.
DigitalOcean Droplet unico	Migracion sencilla y recursos escalables.	No corrige el cuello de botella; puede repetir el mismo problema en una maquina mas grande.	No recomendado como solucion principal para 300+ usuarios.
DigitalOcean Managed PostgreSQL + app separada	Pool de conexiones, backups, failover, metricas y aislamiento de DB.	Costo mensual mayor y requiere migracion/control de PostgREST/Auth si se abandona Supabase completo.	Recomendado si quieren operacion mas estable y menos carga administrativa.
Supabase Cloud Pro/Team	Mantiene Auth/PostgREST/Storage/RLS administrados y reduce operacion.	Costo depende de uso; menos control de bajo nivel.	Muy buena opcion si la prioridad es velocidad de estabilizacion.

Conclusion infraestructura: para 300+ usuarios, la opcion mas sana es separar la base de datos del resto y usar un PostgreSQL administrado o Supabase Cloud. Si se queda self-hosted, debe tratarse como produccion formal: Postgres dedicado, PgBouncer, limites de conexiones, statement_timeout, indices, logs de consultas lentas y alertas.

Tareas Transversales Del Sistema

Prioridad	Tarea
P0	Todas las lecturas a tablas grandes deben tener limit/range, columnas explicitas y filtro por empresa/sucursal.
P0	Eliminar select * en factura, productos2, inventario, usuario, defactura, salida y reportes.
P0	Crear perfiles de columnas: grid, resumen, detalle, impresion, DGII y reporte.
P0	Usar switchMap/debounce/cancelacion en buscadores para evitar multiples requests simultaneos.
P0	Cachear catalogos estables con shareReplay: formas de pago, formas de entrega, ITBIS, permisos, usuario actual, empresa y sucursal.
P0	Revisar RLS de usuario y permisos para que no hagan lecturas secuenciales por fila.
P1	Mover resúmenes pesados a RPC: cierre caja, consulta ventas, 607, inventario sucursal, rendimiento choferes.
P1	Crear índices parciales por filtros reales: pendientes, no impresas, fechas, sucursal, usuario, status.
P1	Agregar auditoria de requests en cliente: pantalla, endpoint, duracion, filas, usuario, sucursal.
P1	Rate limit y proteccion: no permitir doble click de guardar, imprimir, buscar o generar reporte.
P2	Virtual scroll para tablas grandes y exportaciones por job/background cuando superen 5,000 filas.

Matriz Por Pantalla

Cada fila indica el cambio que debe hacer el sistema para reducir consumo en una operacion con mas de 300 usuarios. P0 significa urgente, P1 importante y P2 optimizacion posterior.

Inicio / Home

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/home	Dashboard y resumen inicial	Riesgo medio: usa forkJoin y puede disparar varias consultas al entrar.	Cachear KPIs 30-60 segundos por usuario/sucursal; usar RPC unico para resumen; no recalcular si vuelve a la pantalla; lazy-load graficos.

Facturacion Y Cotizacion

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/facturacion	Factura principal	Riesgo critico: historicamente consultaba muchas facturas y detalles completos.	Ya mejorado: grid limitado y columnas minimas; detalle al seleccionar. Pendiente: terminar eliminar select * en acciones de guardar/imprimir/DGII y aplicar indices de factura.
/private/cotizacion	Cotizaciones	Riesgo alto: patron similar a facturacion.	Ya mejorado: grid con encabezados y detalle bajo demanda. Pendiente: cache de clientes/productos y exportacion por lotes.

Caja

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/caja/CobroFact	Cobro de facturas	Riesgo critico: lista de facturas no impresas/no pagadas y posterior detalle.	Ya mejorado: listado limitado y columnas de caja. Pendiente: boton de refrescar manual, no auto-recargar despues de cada operacion salvo fila afectada, indice parcial para no impresas.
/private/caja/ControlSalida	Control de salida en caja	Riesgo alto: por salida pedia factura completa por cada detalle.	Ya mejorado: ultimo cierre por sucursal, salida pendiente unica, columnas especificas y resumen de factura. Pendiente: reemplazar forkJoin por RPC que devuelva salida + facturas en una sola llamada.
/private/caja/cuadrecaja	Cuadre de caja	Riesgo critico: resumen de pendientes puede leer miles de facturas.	Ya mejorado parcialmente: count planned y uso de RPC si existe. Pendiente: aplicar RPC/indices en BD y calcular totales agregados en SQL.
/private/caja/reciboingreso	Recibo de ingreso	Riesgo medio: historial de recibos.	Ya mejorado: filtro por sucursal, limite 200 y columnas especificas. Pendiente: paginacion real y busqueda por fecha/nombre.

Almacen

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/almacen/entradamerc	Entrada de mercancia	Riesgo alto: productos por codigo en lote y actualizaciones de inventario.	Buscar productos por lote con .in() o RPC; no hacer forkJoin por cada codigo; cachear suplidores/productos recientes; upsert inventario en RPC transaccional.
/private/almacen/controlfact	Control de facturas / despacho almacen	Riesgo critico: consulta factura y detalle para seguimiento.	Usar lista resumida por status/sucursal; detalle bajo demanda; evitar ngOnInit recursivo; indices por fa_despacho, fa_entrega, fa_impalmaf/fa_impalmap.
/private/almacen/ventainterna	Venta interna	Riesgo alto: flujo parecido a factura.	Mismo patron de Facturacion: grid liviano, detalle bajo demanda, productos con debounce, inventario por sucursal con columnas minimas.

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/almacen/pendiente	Pendiente de entrega	Riesgo alto: filtros sobre facturas no entregadas.	RPC paginado por sucursal/status; indice parcial para fa_entrega <> S; no traer detalle hasta seleccionar.
/private/almacen/salidafactura	Salida de factura	Riesgo alto: chofer + facturas + detalles en paralelo.	Consolidar busqueda de facturas asignables; usar RPC para guardar salida y detalles; evitar forkJoin de actualizaciones; bloquear doble guardado.
/private/almacen/devoluciones	Devoluciones	Riesgo alto: toca factura, detalle e inventario.	Transaccion RPC para devolucion; columnas especificas; validar inventario con una sola consulta .in(); indices en devolucion/detalle por sucursal y fecha.
/private/almacen/solicitudprestamo	Solicitud prestamo	Riesgo medio: count exact y select *.	Cambiar a count planned; columnas por grid/detalle; paginacion y filtros por sucursal/fecha/status.

Despacho

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/despacho	Despacho	Riesgo critico: pantalla operativa de facturas, entregas y estados.	Separar tablero por estados con conteos agregados; listar solo 50-100 filas; detalle bajo demanda; cache de despachadores; RPC para marcar despacho/entrega.

Contabilidad

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/contabilidad/facturas-pendientes	Facturas pendientes	Riesgo critico: consulta sobre factura pendiente/cierre/pago.	RPC paginado y agregado; indice por fa_codempr, fa_codsucu, fa_fpago/fa_status/fa_cierre; exportacion por lote.
/private/contabilidad/reporte-607	Reporte 607	Riesgo critico: rangos de fecha y DGII pueden leer muchas facturas.	Ya existe patron RPC en servicio: reforzar como unico camino; generar archivo en backend/job; limitar vista previa; indices por fecha, NCF, empresa.
/private/contabilidad/gastos-menores	Gastos menores	Riesgo bajo-medio.	Paginacion por fecha; columnas especificas; cache de proveedores/formas de pago; indices por fecha/sucursal.
/private/contabilidad/nota-credito	Nota de credito	Riesgo alto: servicio tiene varios select *.	Crear perfiles de columnas; buscar factura origen con resumen; detalle bajo demanda; RPC transaccional para crear NC y actualizar referencias.
/private/contabilidad/encf	ENCF / secuencias fiscales	Riesgo medio-alto: datos criticos fiscales.	Cache de catalogos; consultas por empresa/tipo; indices por codempr/tipo; auditoria de reservas y evitar doble click.

Mantenimientos - Datos Maestros

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/mantenimientos/inventario	Inventario maestro	Riesgo critico: productos2/inventario son top traffic.	Paginacion obligatoria; busqueda minimo 3-4 letras; columnas de grid; indices por codigo, descripcion prefix, grupo, sucursal; virtual scroll.
/private/mantenimientos/inventario-sucursal	Inventario por sucursal	Riesgo critico: lectura cruzada productos/inventario.	Usar RPC resumen_inventario_sucursales/listar_inventario_sucursal; no join client-side; filtros previos obligatorios.
/private/mantenimientos/precios	Precios	Riesgo alto por productos.	No cargar todo el catalogo; buscar por codigo/nombre con debounce; actualizacion por lote en RPC; cache de ultimos productos.

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/mantenimientos/cliente	Clientes	Riesgo medio-alto.	Grid paginado; busqueda por prefijo; columnas minimas; indice por nombre/RNC/telefono; cache de cliente seleccionado.
/private/mantenimientos/sucursales	Sucursales	Riesgo bajo.	Cache global; no recargar por cada pantalla; invalidar solo al guardar.
/private/mantenimientos/supliodor	Suplidores	Riesgo medio.	Paginacion, busqueda con debounce, columnas minimas e indice por nombre/RNC.
/private/mantenimientos/zona	Zonas	Riesgo bajo.	Cache de catalogo; carga una vez por sesion.
/private/mantenimientos/sector	Sectores	Riesgo bajo-medio.	Cache por zona; no cargar todos los sectores si se filtra por zona.
/private/mantenimientos/usuario	Usuarios	Riesgo critico segun reporte: muchas lecturas secuenciales.	Eliminar count exact/select *; indices por correo/idusuario/sucursal; cache de usuario actual; revisar RLS para evitar subconsultas repetidas; paginar permisos.
/private/mantenimientos/choferes	Choferes	Riesgo medio.	Busqueda por sucursal con debounce; columnas minimas; indices por nombre/codigo/sucursal.
/private/mantenimientos/despachadores	Despachadores	Riesgo bajo-medio.	Cache y paginacion; columnas minimas; invalidar cache al guardar.
/private/mantenimientos/Empresas	Empresas	Riesgo bajo.	Cache global y consultas por codigo; evitar select * en auth/perfil.
/private/mantenimientos/grupo-mercancias	Grupos de mercancias	Riesgo bajo.	Cache global; usado por inventario/precios.
/private/mantenimientos/rnc	RNC	Riesgo medio-alto si se consulta padron grande.	Busqueda minimo 5 caracteres; indice por RNC/nombre; no cargar padron completo; cache local de ultimos RNC.

Configuracion

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/mantenimientos/encf	ENCF mantenimiento	Riesgo medio.	Filtros por empresa/tipo; columnas explicitas; operaciones de reserva atomicas.
/private/mantenimientos/modulo	Modulos	Riesgo bajo.	Cachear catalogo de modulos y permisos; cargar una vez por sesion.
/private/mantenimientos/permiso	Permisos	Riesgo alto por forkJoin/permisos.	RPC que devuelva matriz usuario/tipo/modulo en una llamada; cachear catalogos; guardar cambios por lote.
/private/mantenimientos/tipousuario	Tipo usuario	Riesgo bajo.	Cachear y paginar si crece.
/private/mantenimientos/confactura	Contadores factura	Riesgo medio por count exact/select *.	Columnas especificas; bloqueo transaccional/RPC para actualizar contadores; indice por sucursal.
/private/mantenimientos/tasaitbis	ITBIS	Riesgo bajo.	Cache global con shareReplay; columnas explicitas.
/private/mantenimientos/fentrega	Forma entrega	Riesgo bajo.	Cache global; invalidacion al guardar.
/private/mantenimientos/fpago	Forma pago	Riesgo bajo pero muy reutilizada.	Cache global por sesion; no pedir en cada pantalla; map local codigo->descripcion.
/private/mantenimientos/cuentas-bancarias	Cuentas bancarias	Riesgo medio por count exact/select *.	Paginacion, columnas explicitas, cache para cajas y recibos.
/private/mantenimientos/configuracion-global	Configuracion global	Riesgo medio.	Cache por empresa; no recargar en cada factura; invalidar al guardar.

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/mantenimientos/cargar-apk	Carga APK	Riesgo bajo.	Evitar select *; separar metadata de archivo; subir archivos fuera de Postgres si aplica.
/private/mantenimientos/estado-servidor	Estado servidor	Riesgo medio: polling cada 5 segundos.	Polling solo para administradores y con pausa si pestaña oculta; intervalo 15-30s; consolidar metricas en endpoint unico.
/private/mantenimientos/cola-dgii	Cola DGII	Riesgo medio-alto: polling.	Polling adaptativo; limitar filas; filtros por estado/fecha; usar realtime o backoff cuando no hay cambios.

Reportes

Ruta	Pantalla	Riesgo actual	Cambio recomendado
/private/reporte/movproducto	Movimiento producto	Riesgo alto por rango/producto/inventario.	Consulta por filtros obligatorios; RPC paginado; exportacion async; indices por producto, fecha, sucursal.
/private/reporte/consulta-ventas	Consulta ventas	Riesgo critico: facturas por fecha/rango.	RPC agregado por dia/vendedor/sucursal; vista previa limitada; indices por fecha/sucursal/vendedor/status; exportacion por lote.
/private/reporte/ventas-vendedor	Ventas vendedor	Riesgo alto.	RPC de agregados; cache formas pago/vendedores; no descargar facturas para sumar en cliente.
/private/reporte/rendimiento-choferes	Rendimiento choferes	Riesgo alto.	RPC agregado sobre salida/detsalida/factura; filtros obligatorios por fecha/sucursal; indices salida y detsalida.

Plan De Implementacion

Fase	Tiempo	Resultado esperado
Fase 1 - Contencion inmediata	1 a 3 dias	Aplicar indices de Caja/Factura; eliminar select * principales; bajar limites; activar debounce y bloqueo de doble click; monitorear endpoints top.
Fase 2 - Pantallas calientes	1 a 2 semanas	Facturacion, Caja, Despacho, Inventario, Consulta Ventas, Reporte 607 y Usuario con RPCs, paginacion y cache.
Fase 3 - Escala 300 usuarios	2 a 4 semanas	Separar DB/API, connection pooling, statement_timeout, slow query logs, dashboard de performance, pruebas de carga.
Fase 4 - Operacion madura	Mensual	Revisar indices, vacuum/analyze, trazas por pantalla, alertas de CPU/temperatura/timeout y presupuesto de consultas por release.

Criterios De Aceptacion Para Cliente

- Abrir Facturacion, Caja, Despacho e Inventario no debe superar 2-3 solicitudes pesadas por pantalla.
- Ninguna tabla operativa debe cargar mas de 100 filas iniciales salvo exportacion confirmada.
- Las busquedas deben iniciar con minimo de caracteres y cancelar la consulta anterior.
- Los reportes deben usar RPC/consulta agregada y no sumar miles de filas en el navegador.
- PostgreSQL debe mantenerse por debajo de 60-70% CPU sostenido en hora pico; picos breves son aceptables.
- Timeouts 504 deben tender a cero bajo uso normal; cualquier query mayor de 2 segundos debe quedar registrada.

Fuentes y notas

DigitalOcean Managed Databases: documentacion oficial de PostgreSQL, connection pooling/PgBouncer, metricas, failover y precios consultada el 24/08/2026. Supabase Self-hosting: documentacion oficial sobre Docker, produccion con HTTPS/reverse proxy y consideraciones de analiticas self-hosted consultada el 24/08/2026.

Este documento no sustituye una prueba de carga. La recomendacion final debe validarse con mediciones despues de aplicar indices, RPCs y cambios de cliente.